

Manual Técnico

Sistema Sudoku – USAC – LFP

Introducción

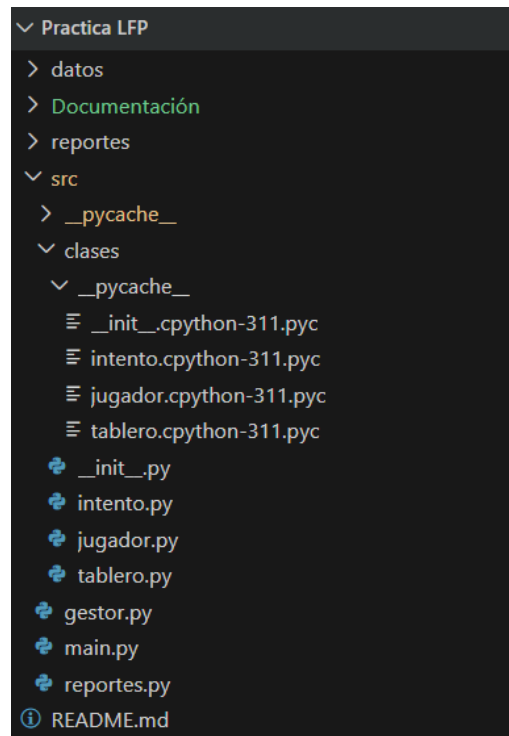
El manual técnico proporciona la documentación de la arquitectura del programa, patrones de diseño, diagrama de clases, detalles de implementación de clases y métodos y algoritmos clave utilizados en el desarrollo de la aplicación.

1. Arquitectura del sistema

El sistema está diseñado bajo una arquitectura por capas / MVC (Modelo-Vista-Controlador) desacoplada, implementada en Python

- **Modelo (src/clases/)**: Contiene la representación orientada a objetos de las entidades del dominio (Tablero, Jugador, Intento), con sus reglas de validación y estado.
- **Controlador (gestor.py)**: Encargado del parseo de archivos .lfp, la manipulación de colecciones en memoria y la coordinación del proceso de calificación.
- **Vista / Generador (main.py y reportes.py)**: Maneja el menú e interfaz por consola, así como la construcción dinámica de las plantillas HTML para la salida de reportes.

2. Estructura de directorios



3. Diagramas de Clases UML

classDiagram

```
class Tablero {  
    +str id_sudoku  
    +str dificultad  
    +str cadena_original  
    +list matriz  
    +dict pistas  
    +_string_a_matriz(cadena) list  
    +_obtener_pistas() dict  
}
```

```
class Jugador {  
    +str carnet  
    +str nombres  
    +str apellidos  
    +str nivel  
    +list intentos  
    +nombre_completo() str  
}
```

```
class Intento {  
    +str carnet  
    +str id_sudoku  
    +str solucion_str  
    +int tiempo_segundos  
    +str fecha  
    +list matriz_solucion  
    +float porcentaje_validez  
    +bool pistas_respetadas  
    +_string_a_matriz(cadena) list  
    +calificar(tablero_original) float  
    +_verificar_pistas(tablero_original) bool  
    +_validar_filas() int  
    +_validar_columnas() int  
    +_validar_cajas() int  
    +resuelto_correctamente() bool  
}
```

```
class GestorTorneo {  
    +dict tableros  
    +dict jugadores  
    +list intentos
```

```

+cargar_sudokus(ruta) bool
+cargar_jugadores(ruta) bool
+cargar_intentos(ruta) bool
+calificar_intentos() int
}

class GeneradorReportes {
+str ruta_destino
+generar_reporte_ranking(jugadores) str
+generar_reporte_intentos(intentos, jugadores) str
}

GestorTorneo "1" *-- "*" Tablero
GestorTorneo "1" *-- "*" Jugador
GestorTorneo "1" *-- "*" Intento
Jugador "1" o-- "*" Intento
Intento ..> Tablero : valida contra
GeneradorReportes ..> Jugador : lee
GeneradorReportes ..> Intento : lee

```

4. Descripción de Módulos y Clases

4.1. Módulo src/clases/tablero.py

Representa una plantilla inicial de Sudoku cargada desde el archivo sudokus.lfp.

- Atributos:
 - id_sudoku (str): Identificador único del tablero (ej. SUD01).
 - dificultad (str): Nivel de dificultad asignado (Fácil, Medio, Difícil).
 - cadena_original (str): Cadena de 81 caracteres con la configuración inicial.
 - matriz (list): Representación bidimensional 9×9 de los valores.
 - pistas (dict): Diccionario con claves (fila, columna) y valores int que representan los números fijos no modificables.

4.2. Módulo src/clases/jugador.py

Almacena la información de los usuarios que participan en el torneo.

- Atributos:
 - carnet (str): Identificador numérico del estudiante.

- o nombres (str) / apellidos (str): Nombres del jugador.
 - o nivel (str): Categoría del jugador (Principiante, Intermedio, Avanzado).
 - o intentos (list): Lista de objetos Intento asociados a este jugador.
- Métodos:
 - o nombre_completo(): Retorna la concatenación de nombres y apellidos.

4.3. Módulo src/clases/intento.py

Representa una resolución sometida por un jugador y contiene el algoritmo de evaluación.

- Atributos:
 - o carnet (str), id_sudoku (str), solucion_str (str), tiempo_segundos (int), fecha (str).
 - o matriz_solucion (list): Matriz 9×9 construida a partir de solucion_str.
 - o porcentaje_validez (float): Punteo asignado entre 0.0% y 100.0% .
 - o pistas_respetadas (bool): Indicador de si el intento conservó los dígitos fijos.
- Métodos Clave:
 - o _string_a_matriz(cadena): Convierte una cadena de 81 caracteres en una matriz de 9×9 .
 - o calificar(tablero_original): Ejecuta la lógica de calificación y retorna la validez calculada.
 - o _verificar_pistas(tablero_original): Valida que las posiciones definidas en tablero_original.pistas coincidan exactamente con matriz_solucion.
 - o _validar_filas(), _validar_columnas(), _validar_cajas(): Retornan el conteo de subestructuras que satisfacen el conjunto $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

4.4. Módulo src/gestor.py (GestorTorneo)

Actúa como la fachada principal de la lógica del negocio.

- Atributos:
 - o tableros (dict): {id_sudoku: Objeto Tablero}
 - o jugadores (dict): {carnet: Objeto Jugador}
 - o intentos (list): Lista ordenada de objetos Intento
- Métodos Clave:
 - o cargar_sudokus(ruta): Parsea el archivo .lfp separado por comas, ignorando líneas inválidas o cadenas con longitud diferente de 81.
 - o cargar_jugadores(ruta): Instancia la colección de jugadores.
 - o cargar_intentos(ruta): Carga los intentos y los asocia bidireccionalmente a la lista intentos de cada Jugador.

- `calificar_intentos()`: Itera sobre la lista de intentos y ejecuta `.calificar(tablero)` utilizando el tablero correspondiente.

4.5. Módulo `src/reportes.py` (`GeneradorReportes`)

Módulo encargado de persistir los resultados procesados en formato HTML.

- Métodos:
 - `generar_reporte_ranking(jugadores)`: Ordena a los jugadores según la función de ordenamiento multicriterio y genera la tabla HTML.
 - `generar_reporte_intentos(intentos, jugadores)`: Genera el registro cronológico completo de partidas.

5. Algoritmos y lógica

5.1. Conversión de Cadenas de 81 Caracteres a Matriz 9×9

```
def _string_a_matriz(self, cadena):
    matriz = []
    for i in range(9):
        fila = []
        for j in range(9):
            caracter = cadena[i * 9 + j]
            val = int(caracter) if caracter.isdigit() else 0
            fila.append(val)
        matriz.append(fila)
    return matriz

def _extraer_pistas(self):
    pistas = {}
    for i in range(9):
        for j in range(9):
            if self.matriz[i][j] != 0:
                pistas[(i, j)] = self.matriz[i][j]
    return pistas
```

6. Manejo de excepciones

- **Archivos no encontrados (FileNotFoundError):** Capturados en la interfaz de consola notificando al usuario para que verifique la ruta ingresada.
- **Resiliencia en Lectura:** El parser de archivos .lfp valida la cantidad de campos esperados por fila antes de instanciar los objetos. Si una línea no cumple con los requisitos, se ignora evitando que el programa colapse.
- **Escalabilidad:** Las estructuras de datos principales son diccionarios basados en tablas Hash (dict), lo que permite búsquedas de tableros y jugadores en tiempo constante $O(1)$.